Trappist Tour: dizajni dhe analiza e algoritmeve për një problem të optimizimit shumë-objektiv të trajektoreve hapësinore

Projekt në lëndën Dizajni dhe Analiza e Algoritmeve

Diona Gerxhaliu (diona.gerxhaliu@student.uni-pr.edu)

Universiteti i Prishtinës, 2026

Përmbajtja

1 Hyrje **2**

2 Formulimi i problemit **2**
2.1 Vektori i vendimeve . . . 2
2.2 Modeli fizik (si e llogarit kodi zyrtar) . . . 3
2.3 Metrika: hipervëllimi . . . 3

3 Implementimi dhe verifikimi i modelit **4**

4 Analiza e strukturës së problemit **4**

5 Algoritmet **6**
5.1 A1 – Kërkim i rastësishëm (bazë) . . . 6
5.2 A2 – NSGA-II (algoritëm evolucionar gjenerik shumë-objektiv) . . . 6
5.3 A3 – Greedy . . . 6
5.4 A4 – Beam search shumë-objektiv me dekompozim sipas hapave (algoritmi kryesor) . . . 6
5.5 A5 – CMA-ES për lustrimin lokal . . . 7
5.6 A6 – Zgjedhja optimale e ≤ 100 zgjidhjeve (programim dinamik) . . . 7
5.7 Pipeline-i i plotë . . . 8

6 Eksperimentet **8**
6.1 Krahasimi i algoritmeve . . . 9
6.2 Shkallëzimi sipas gjerësisë së beam-it . . . 10
6.3 Ablacionet . . . 10

7 Rezultati final **10**

8 Përfundime **11**

9 Si të ekzekutohet kodi **13**

10 Referencat **13**

1 Hyrje

Problemi **Trappist Tour** është pjesë e garës *Space Optimisation Competition* (SpOC), të organizuar nga Ekipi i Koncepteve të Avancuara (ACT) i Agjencisë Hapësinore Evropiane (ESA) bashkë me konferencën GECCO 2022. Problemi zyrtar dhe kodi i vlerësimit gjenden në platformën *Optimize*: https://optimize.esa.int/challenge/spoc-trappist-tour/About.

Një sondë hyn në sistemin yjor TRAPPIST-1 nga një sferë me rreze 10 AU dhe duhet të vizitojë **secilin nga 7 planetët (b, c, d, e, f, g, h) saktësisht një herë**, në një renditje të zgjedhur lirisht. Trajektorja mund të ndryshohet vetëm në dy mënyra:

- me **asistencë gravitacionale** (*fly-by*) pranë një planeti, pa kosto karburanti;
- me **manovra impulsive në hapësirë të thellë** (*Deep Space Manoeuvre*, DSM), të cilat kushtojnë ndryshim shpejtësie ΔV, pra karburant.

Kërkohen dy objektiva në konflikt: ΔV **total minimal** dhe **kohë totale e misionit minimale**. Prandaj nuk ekziston një zgjidhje e vetme optimale, por një **front Pareto** zgjidhjesh kompromisi. Cilësia e një bashkësie prej ≤ 100 zgjidhjesh matet me **hipervëllimin** (*hypervolume*) në lidhje me pikën referuese (2500 m/s, 4000 ditë).

Nga këndvështrimi i lëndës, problemi është interesant sepse kombinon:

1. një pjesë **kombinatorike** (renditja e planetëve: $7! = 5040$ permutacione),
2. një pjesë **të vazhdueshme**, jolineare dhe me shumë minimume lokale (27 ndryshore reale),
3. **dy objektiva**, dhe
4. kufizime që janë jo-të-lëmuara (kontrolli i afrimit me yllin).

Në këtë projekt kemi projektuar, implementuar dhe analizuar gjashtë algoritme, nga ato gjenerike (kërkim i rastësishëm, NSGA-II), te ato që shfrytëzojnë strukturën e problemit (kërkim *greedy*, *beam search* shumë-objektiv me dekompozim sipas hapave, CMA-ES lokal dhe programim dinamik për zgjedhjen optimale të nënbashkësisë).

Rezultati kryesor. Pipeline-i i propozuar arrin hipervëllim **5,635,615** (rezultati në platformë: $-5,635,615$) me 40 zgjidhje, që do ta vendoste në **vendin e 3-të** të renditjes zyrtare të SpOC 2022. Algoritmet gjenerike, me buxhet të krahasueshëm llogaritës, nuk gjejnë asnjë zgjidhje brenda pikës referuese (hipervëllim 0).

2 Formulimi i problemit

2.1 Vektori i vendimeve

Kodi zyrtar përdor një vektor me 34 komponentë:

Tabela 1: Vektori i vendimeve (34 komponentë).

Pjesa	Ndryshoret	Kuptimi	Kufijtë
hyrja	u, v	drejtimi i pikës së hyrjes në sferën 10 AU	$u \in [0,1]$, $v \in [0.25, 0.75]$
	T_0	kohëzgjatja e hapit të hyrjes (ditë)	$[5, 2000]$
6 hapa	β_k	këndi i planit të fly-by	$[0, 2\pi]$
	$r_{p,k}/R$	distanca e periapsit të fly-by (në rreze të Tokës)	$[1.1, 100]$
	η_k	pjesa e hapit pas së cilës bëhet DSM	$[0.001, 0.999]$
	T_k	kohëzgjatja e hapit (ditë)	$[5, 2000]$
renditja	$s_0, \dots, s_6$	permutacion i $\{0, \dots, 6\}$	numra të plotë

2.2 Modeli fizik (si e llogarit kodi zyrtar)

Lëvizja modelohet si problem i dy trupave rreth yllit ($\mu = GM_\star$). Për çdo vektor x:

1. **Hyrja.** Nga pika në sferë deri te planeti s_0 zgjidhet *problemi i Lambert-it* (gjej orbitën që lidh dy pozicione në një kohë të dhënë). Kosto: $\Delta V_{\text{init}} = |\|v_1\| - 10 \text{ km/s}|$ (sonda nis me 10 km/s).
2. **Për çdo hap** $k = 1..6$**:** fly-by te planeti s_{k-1} (rrotullon shpejtësinë relative me një kënd $\delta(r_p)$ rreth një drejtimi të caktuar nga β); lundrim kepler-ian për $\eta_k T_k$ ditë; DSM; pastaj hark Lambert për $(1 - \eta_k)T_k$ ditë deri te planeti s_k. Kosto e hapit është madhësia e DSM.
3. **Objektivat:** $f_1 = \Delta V_{\text{init}} + \sum_k \text{DSM}_k$ (m/s) dhe $f_2 = \sum_{k=0}^{6} T_k$ (ditë).
4. **Kufizimet:** renditja duhet të jetë permutacion; dhe në çdo hark që kalon periapsin, distanca minimale nga ylli duhet të jetë $\geq 834\,840$ km (10 rreze ylli).

2.3 Metrika: hipervëllimi

Për një bashkësi pikash P në hapësirën e objektivave dhe pikë referuese r, $\text{HV}(P)$ është sipërfaqja e zonës që dominohet nga P dhe dominon r. Për dy objektiva, nëse pikat jo-të-dominuara renditen sipas f_1:

$$\text{HV}(P) = \sum_i \left(f_1^{(i+1)} - f_1^{(i)}\right)\left(r_2 - f_2^{(i)}\right), \qquad f_1^{(n+1)} := r_1.$$

Vlera maksimale teorike është $2500 \times 4000 = 10^7$. Platforma raporton $-\text{HV}$.

3 Implementimi dhe verifikimi i modelit

Kodi zyrtar varet nga bibliotekat `pykep` dhe `pygmo`, të cilat shpërndahen vetëm si binare (conda-forge). Për ta bërë projektin të pavarur, **rutinat astrodinamike u rishkruan në Python/NumPy** (`trappist/astro.py`) duke ndjekur kodin burimor C++ të `pykep` 2.6:

- zgjidhësi i Lambert-it i Izzo-s (2015) me iteracione Householder;
- propagimi kepler-ian me koeficientët e Lagrange-it (ekuacioni i Kepler-it për elipsë dhe hiperbolë);
- fly-by (`fb_prop`), konvertimet element orbital ↔ pozicion/shpejtësi;
- funksioni i fitnesit (`trappist/problem.py`) **rresht për rresht si ai zyrtar**.

Kodi zyrtar ruhet i paprekur në `official/trappist_tour_official.py`.

Verifikimi (`tests/`). Çdo rutinë krahasohet me një metodë **të pavarur**: integrim numerik i ekuacioneve të lëvizjes me SciPy (DOP853, tolerancë 10^{-12}). Gabimet relative janë $< 10^{-7}$. Zgjidhja shembull e ESA-s jep të njëjtat vlera ($T = 3469.8267$ ditë, $\Delta V = 2378.99$ m/s, e lejuar). Versionet e vektorizuara (NumPy) krahasohen me ato skalare.

Vërejtje mbi kodin zyrtar. Gjatë analizës vumë re disa veçori që ndikojnë drejtpërdrejt në dizajnin e algoritmeve (dhe që i ruajtëm qëllimisht, sepse serveri vlerëson me këtë kod):

1. Për harqe **hiperbolike** periapsi llogaritet si $-a(1-e) < 0$, kështu që *çdo* hark hiperbolik që kalon periapsin shkel kufizimin. Pasojë: hapi i hyrjes duhet ta takojë planetin e parë **para** periapsit, dhe fly-by i parë duhet ta "kapë" sondën në orbitë eliptike.
2. Perioda e një harku hiperbolik del `NaN`, kështu që testi `dt > period` është gjithmonë i rremë.
3. Në rastin e "death penalty" funksioni kthen 6 vlera në vend të 4 (gabim i vogël, rrallë aktiv).

4 Analiza e strukturës së problemit

Para se të zgjidhnim algoritmet, analizuam problemin. Këto vëzhgime e drejtojnë dizajnin.

(a) Hapësira e kërkimit. 5040 renditje × $\mathbb{R}^{27}$. Kërkimi shterues është i pamundur; edhe me vetëm 10 vlera për çdo ndryshore të vazhdueshme do të kishim $5040 \cdot 10^{27}$ kombinime.

(b) Hapi i hyrjes është praktikisht i ndarë. ΔV_{init} varet pothuajse vetëm nga T_0 (Figura 1): është ≈ 0 për $T_0^* \approx 1500$ ditë dhe rritet ≈ 8 m/s për çdo ditë të fituar. Pra, rreth 1500 ditë të misionit janë "të detyrueshme" dhe pjesa interesante është turi me 6 hapa.

(c) Kapja dhe "pompimi" i orbitës. Pas fly-by të parë sonda mbetet në një orbitë shumë ekscentrike ($e \approx 0.95$, perioda 150–350 ditë). Një hap i shkurtër është i pamundur pa kosto të madhe, sepse sonda duhet të bëjë rreth një rrotullim. Fly-by-t e njëpasnjëshëm mund ta **zvogëlojnë periodën** (p.sh. $210 \to 120 \to 80$ ditë) pa karburant.

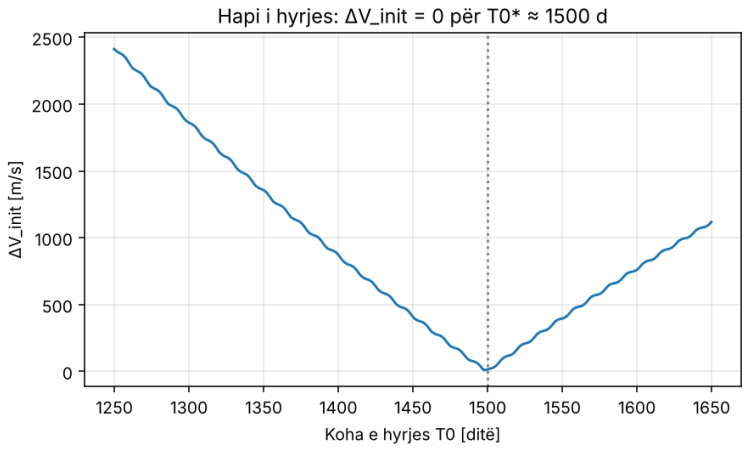


Figura 1: Kostoja e hapit të hyrjes në funksion të T_0.

Kjo shpjegon pse një algoritëm *greedy* që zgjedh hapin më të lirë tani nuk mjafton: duhet *lookahead*.

(d) Kosto e një hapi ka formë "V". Nëse DSM vendoset menjëherë pas fly-by, kosto e hapit $i \to j$ me kohëzgjatje T është mospërputhja mes shpejtësisë që kërkon harku Lambert dhe asaj që mund të japë fly-by. Ajo bëhet **saktësisht zero** kur $\|v_{\text{out,rel}}(T)\| = \|v_{\text{in,rel}}\|$ (zgjidhje *balistike*) dhe rritet shumë shpejt rreth saj (Figura 2). Një rrjet i thjeshtë në T humb qindra m/s; prandaj çdo minimum lokal përmirësohet me **kërkim me prerje të artë** (golden section).

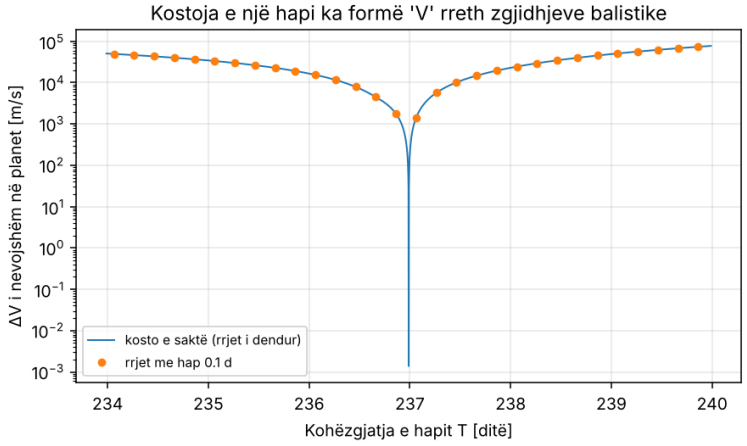


Figura 2: Kosto e një hapi rreth një zgjidhjeje balistike: rrjeti me hap 0.1 ditë e humb minimumin.

(e) Kondicionim shumë i keq. Ndryshimi i kohës së një hapi me 1 sekondë ndryshon ΔV me metra/sekondë, nëse parametrat e fly-by nuk ndryshojnë njëkohësisht. Zgjidhjet e mira janë në "lugina" shumë të ngushta. Metodat me gradient (provuam SLSQP) dështojnë; CMA-ES, që mëson formën e luginës, funksionon.

(f) Fronti përbëhet nga ture të ndryshme. Ndryshimi i kohës totale me pak ditë ndryshon fazat e të gjithë planetëve (periodat janë 1.5–19 ditë), prandaj fronti Pareto

nuk është një kurbë e vetme që mund të ndiqet me *continuation*, por bashkësi turesh diskrete. Kjo e bën **diversitetin e kërkimit** po aq të rëndësishëm sa optimizimin lokal.

5 Algoritmet

5.1 A1 – Kërkim i rastësishëm (bazë)

Gjenerojmë N vektorë uniformisht brenda kufijve (renditja si permutacion i rastësishëm) dhe mbajmë pikat e lejuara jo-të-dominuara.

- Kompleksiteti: $O(N)$ vlerësime + $O(N \log N)$ për frontin.
- Qëllimi: të tregojë sa i vështirë është problemi pa njohuri për strukturën.

5.2 A2 – NSGA-II (algoritëm evolucionar gjenerik shumë-objektiv)

Implementim i plotë i NSGA-II (Deb et al., 2002) për kodimin e përzier:

- pjesa reale (27 gjene në $[0, 1]$): kryqëzim SBX + mutacion polinomial;
- pjesa permutacion (7 gjene): kryqëzim OX1 + mutacion me shkëmbim;
- kufizimet: *constrained domination* (zgjidhjet e lejuara para të palejuarave);
- seleksion me turne binar sipas (rangut, distancës së grumbullimit).

Kompleksiteti për gjeneratë: $O(N)$ vlerësime + $O(MN^2)$ për renditjen jo-të-dominuar ($M = 2$ objektiva, N = popullsia).

5.3 A3 – Greedy

Ndërton turin hap pas hapi: në çdo nivel zgjedh vetëm fëmijën më të mirë (beam search me gjerësi 1). Kompleksiteti: $O(n^2)$ zgjerime, ku $n = 7$. Shërben si rast kufitar i A4.

5.4 A4 – Beam search shumë-objektiv me dekompozim sipas hapave (algoritmi kryesor)

Ideja. Turi ndërtohet si një pemë kërkimi: niveli 0 janë hyrjet (planeti i parë × drejtimi (u, v) në një rrjet 24×7, me T_0^* optimal), niveli k janë turet e pjesshme pas k fly-by-ve. Çdo nyje ruan: renditjen, epokën, shpejtësinë e mbërritjes, ΔV e grumbulluar dhe kohën e grumbulluar. Kjo **dekompozon** problemin 27-dimensional në 6 nënprobleme 4-dimensionale (një për hap), me gjendjen e mbërritjes si "kujtesë" mes tyre.

Gjenerimi i fëmijëve për nyjen në planetin i dhe çdo planet të pavizituar j:

1. *Lambert screening* (i shpejtë): për ~8 000 kohëzgjatje T në një rrjet me hap 0.04 ditë, zgjidhet Lambert-i i vektorizuar dhe llogaritet analitikisht kostoja e fly-by optimal (ligji i kosinusit). Minimumet lokale përmirësohen me prerje të artë → gjen zgjidhjet **balistike**.
2. *Differential Evolution* (i saktë): për çdo dritare kohore $[5, 30], [30, 60], \ldots, [250, 340]$ ditë optimizohen (β, r_p, η, T) në modelin zyrtar. Popullsitë e të gjitha detyrave $(j, \text{dritare})$ bashkohen në **një matricë të vetme** dhe vlerësohen me një thirrje NumPy për gjeneratë. DE niset me zgjidhjet nga hapi 1.
3. Çdo kandidat rivlerësohet me modelin e saktë; mbahen fëmijët **jo-të-dominuar** sipas (ΔV, T) për çdo j.

Seleksioni (beam). Nga të gjithë fëmijët e nivelit mbahen vetëm B. Për të ruajtur diversitetin dhe karakterin shumë-objektiv:

- përdoren disa shuma të peshuara $\Delta V + \lambda\,(T + h)$, me $\lambda \in \{0, 1, 2, 4, 8, 16\}$ m/s/ditë dhe $B/6$ nyje për secilën ("feta" të frontit);
- h = (hapa të mbetur) × (perioda e orbitës aktuale) është një vlerësim i kohës që mbetet, në frymën e algoritmit A*; ai shpërblen nyjet që kanë "pompuar" orbitën;
- lejohen maksimum $B/20$ nyje për çdo çift (bashkësia e vizituar, planeti aktual).

```
BEAM-SEARCH(roots, B):
  L ← SELECT(roots, B)
  for depth = 1..6:
      C ← ∅
      for each node in L:                 # paralel në 2 bërthama
          C ← C ∪ EXPAND(node)            # screening + DE + front lokal Pareto
      L ← SELECT(C, B)  (në nivelin 6: L ← C)
  return L                                # ture të plota

SELECT(C, B):
  S ← ∅
  for λ in Λ:
      sort C by ΔV + λ·(T + h)
      add the first B/|Λ| nodes not in S, respecting the per-key cap
  return S
```

Kompleksiteti. Le të jenë $n = 7$ planetët, m madhësia e rrjetit kohor, W numri i dritareve, $N_P \cdot G$ buxheti i DE. Një zgjerim kushton $O\big(n\,(m + W N_P G)\big)$ vlerësime hapi, prandaj i gjithë kërkimi kushton

$$O\big(n \cdot B \cdot n\,(m + W N_P G)\big) = O\big(n^2 B\,(m + W N_P G)\big),$$

pra **linear në** B dhe polinomial në n, në vend të $O(n!)$ të numërimit shterues të renditjeve. Kujtesa: $O(B)$ nyje për nivel. Seleksioni kushton $O(|C| \log |C| \cdot |\Lambda|)$.

5.5 A5 – CMA-ES për lustrimin lokal

Për çdo tur të plotë optimizojmë njëkohësisht të 27 ndryshoret (renditja fikse) me CMA-ES (Hansen & Ostermeier, 2001), implementuar nga zeroja (`trappist/local.py`), me formulim ε-constraint: minimizo ΔV me kusht $\sum T_k \le$ buxheti. Për shkak të kondicionimit të keq, hapi fillestar është i ndryshëm për çdo ndryshore (~ 1 s për kohët e hapave, $\sim 10^{-5}$ për të tjerat). Kompleksiteti: $O(E \cdot d^2)$ për E vlerësime në $d = 27$ dimensione (përditësimi i matricës së kovariancës), praktikisht i dominuar nga E vlerësime të fitnesit.

5.6 A6 – Zgjedhja optimale e ≤ 100 zgjidhjeve (programim dinamik)

Platforma pranon maksimum 100 vektorë, kurse algoritmet prodhojnë qindra pika jo-të-dominuara. Duhet zgjedhur nënbashkësia me hipervëllim maksimal (*Hypervolume Subset Selection Problem*). Në 2 dimensione ky problem ka **nënstrukturë optimale**: renditim pikat sipas f_1; nëse zgjedhim $i_1 < \cdots < i_k$, kontributi i pikës i_m është

drejtkëndëshi $(f_1^{(i_{m+1})} - f_1^{(i_m)})(r_2 - f_2^{(i_m)})$ dhe varet **vetëm nga pika e ardhshme e zgjedhur**. Prandaj

$$\mathrm{best}_c(i) = \max_{j>i} \Big[(f_1^{(j)} - f_1^{(i)})(r_2 - f_2^{(i)}) + \mathrm{best}_{c-1}(j) \Big], \qquad \mathrm{best}_1(i) = (r_1 - f_1^{(i)})(r_2 - f_2^{(i)}).$$

Kompleksiteti $O(k\,n^2)$ kohë dhe $O(k\,n)$ kujtesë ($k = 100$). Korrektësia u verifikua kundrejt kërkimit shterues të të gjitha nënbashkësive në teste të rastësishme (`python -m trappist.hv`).

5.7 Pipeline-i i plotë

```
entry_nodes  →  A4 beam search (disa ekzekutime)  →  bashkimi i tureve
             →  A5 CMA-ES në 3 shtresat e para jo-të-dominuara
             →  A6 nënbashkësia optimale (≤ 100)  →  submission.json
```

6 Eksperimentet

Mjedisi. Python 3.13, NumPy, SciPy; 2 bërthama CPU. Çdo vlerësim i fitnesit kushton ≈ 0.8 ms në Python të pastër; vlerësimi i vektorizuar i një hapi ≈ 3–10 μs.

Tabela 2 përmbledh të gjitha ekzekutimet. Algoritmet e rastësishme janë ekzekutuar me 3 farë (*seeds*), beam search-i është determinist për një farë të dhënë.

Tabela 2: Krahasimi i algoritmeve (HV = hipervëllimi, sa më i madh aq më mirë; T min = koha më e shkurtër ndër zgjidhjet me $\Delta V < 2500$ m/s).

Algoritmi	HV	ΔV min [m/s]	T min* [d]	# të lejuara	Koha [s]	Ekz.
A1 Kërkim i rastësishëm (100k vlerësime, 3 farë)	0	32,596.9	–	7	198	3
A2 NSGA-II (100k vlerësime, 3 farë)	0	6,141.9	–	18,160	227	3
A3 Greedy (B = 1)	0	–	–	0	6	1
A4 Beam search, B = 10	5,225,697	36.6	1,866.2	25	67	1
A4 Beam search, B = 30	5,318,222	36.6	1,780.9	76	179	1
A4 Beam search, B = 50	5,408,991	17.2	1,771.3	127	251	1
A4 Beam search, B = 75	5,418,733	14.5	1,710.4	207	382	1
A4 Beam search, B = 150	5,482,582	9.2	1,710.4	364	769	1
A4 Beam search, B = 300	5,572,792	9.6	1,691.1	687	1,389	1
A4 vetëm Lambert-screening, B = 50	0	–	–	0	78	1
A4 vetëm DE, B = 50	5,388,159	40.2	1,703.3	90	162	1
A4 B = 150, T0 i zhvendosur (−40, −80 d)	3,747,060	710.5	1,821.4	364	849	1

Algoritmi	HV	ΔV min [m/s]	T min* [d]	# të lejuara	Koha [s]	Ekz.
A4 B = 300, farë 3, DE G = 90, T0 (0, −20 d)	5,626,487	0.1	1,689.6	677	1,818	1
Pipeline i plotë A4+A5+A6 (40 zgjidhje)	**5,635,615**	0.1	1,689.6	40	–	1

6.1 Krahasimi i algoritmeve

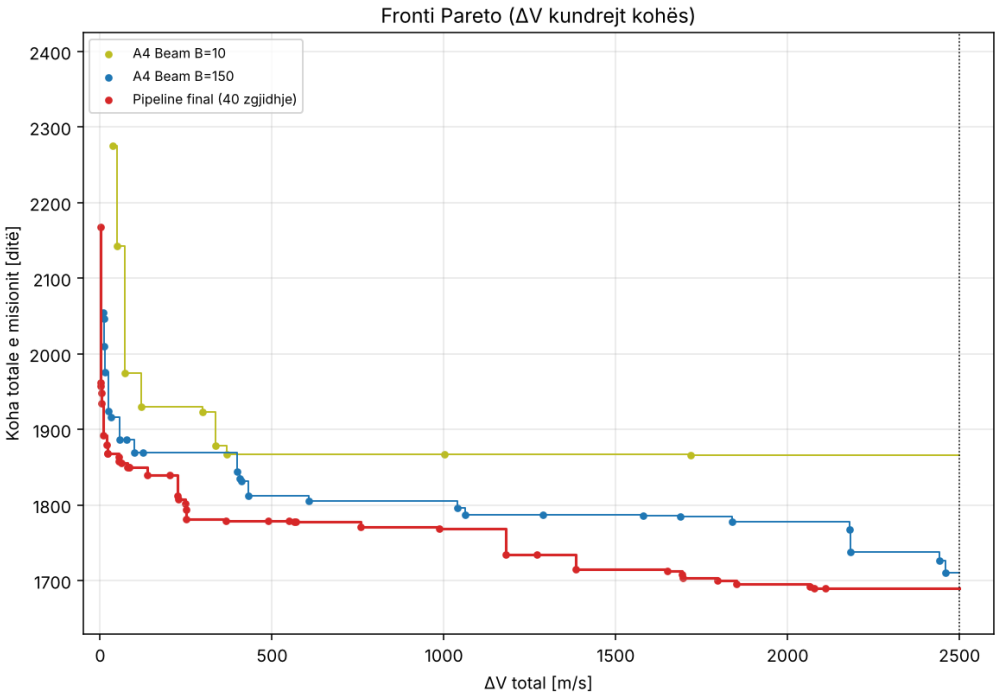


Figura 3: Frontet Pareto (zmadhuar). A1, A2 dhe A3 nuk kanë asnjë pikë brenda pikës referuese, prandaj nuk shfaqen.

- **Kërkimi i rastësishëm** gjen mesatarisht vetëm 7 zgjidhje të lejuara në 100 000 vlerësime, dhe më e mira ka ΔV = 32.6 km/s, 13 herë mbi kufirin prej 2.5 km/s. Kjo tregon se zgjidhjet e mira zënë një pjesë jashtëzakonisht të vogël të hapësirës.
- **NSGA-II** gjen shumë zgjidhje të lejuara (mesatarisht 18,160), por më e mira ka ΔV = 6.1 km/s, pra hipervëllimi mbetet 0. Operatorët gjenerikë (SBX, OX1) nuk mund t'i ndjekin luginat e ngushta të zgjidhjeve balistike: ndryshimi i një kohe me pak sekonda pa ndryshuar fly-by-n njëkohësisht prish gjithë turin (shih seksionin 4e). Edhe renditja e planetëve ndryshon kuptimin e të gjitha gjeneve të tjera, prandaj kryqëzimi i dy prindërve të mirë rrallë jep fëmijë të mirë.
- **Greedy** (B = 1) dështon plotësisht: zgjedh hyrjen me shpejtësinë relative më të vogël (në dukje më e mira), por nga ajo gjendje asnjë hap i mëtejshëm nuk përshtatet brenda buxhetit prej 2500 m/s. Mungesa e kthimit prapa (*backtracking*) e çon në rrugë qorre.

- **Beam search** me vetëm B = 10 jep tashmë 5,225,697 brenda 67 s. Kjo konfirmon se dekompozimi sipas hapave, jo fuqia llogaritëse, është çelësi.

6.2 Shkallëzimi sipas gjerësisë së beam-it

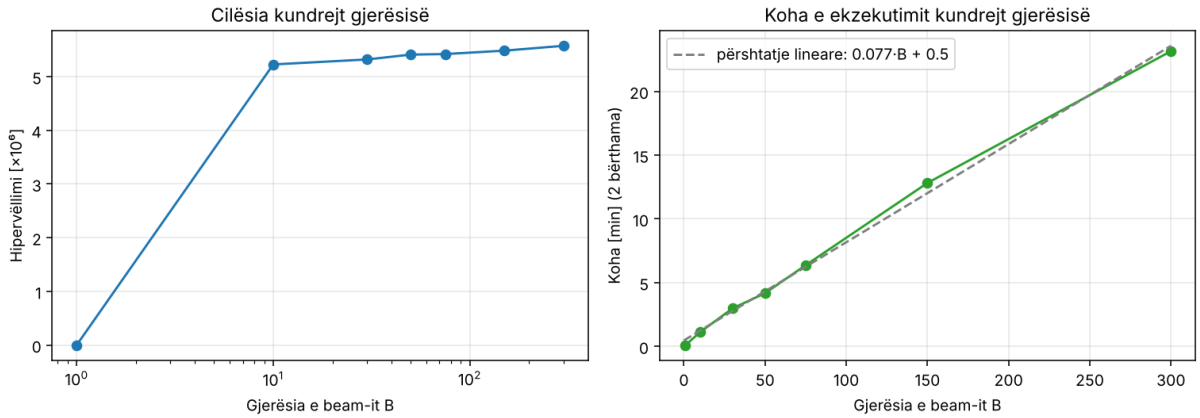


Figura 4: Hipervëllimi dhe koha e ekzekutimit në funksion të gjerësisë B.

Koha e ekzekutimit rritet **linearisht** me gjerësinë: përshtatja $t(B) \approx 4.58\,B + 37$ s ka $R^2 = 0.998$, siç parashikon analiza teorike $O(n^2 B(m + W N_P G))$. Cilësia rritet në mënyrë monotone por me kthim zvogëlues (*diminishing returns*): nga B = 150 në B = 300 koha dyfishohet (13 → 23 min) ndërsa HV rritet me 1.6%. Kjo është sjellja tipike e beam search-it: gjerësia kontrollon kompromisin mes cilësisë dhe kohës.

6.3 Ablacionet

Për të kuptuar rolin e secilit komponent të gjenerimit të fëmijëve, me B = 50:

- **Vetëm Lambert-screening** (pa DE): **asnjë tur i plotë.** Modeli me DSM te planeti është shumë pesimist kur fly-by nuk mund ta kthejë shpejtësinë mjaftueshëm (planete të lehta ose shpejtësi relative e madhe): atëherë DSM duhet bërë larg, pranë apoapsit, ku kushton shumë më pak. Në këto raste kostoja e vlerësuar ishte dhjetëra km/s, kurse në modelin e saktë 60–130 m/s.
- **Vetëm DE** (pa screening): HV = 5,388,159 në 162 s, që është 0.4% më pak se hibridi (5,408,991). DE vetë gjen hapa të mirë, por zgjidhjet balistike janë minimume shumë të ngushta që DE i gjen vetëm rastësisht.
- **Hibridi** kombinon të dyja: screening jep pika nisjeje balistike (0 m/s) për DE, dhe DE trajton rastet ku duhet DSM në hapësirë të thellë.

Përfundimi i ablacionit: komponentët plotësojnë njëri-tjetrin, dhe DE në modelin e saktë është komponenti i domosdoshëm.

7 Rezultati final

Pipeline-i final bashkon turet e plota nga të gjitha ekzekutimet e beam search-it, i lustron me CMA-ES dhe zgjedh me programim dinamik nënbashkësinë optimale. Rezultati:

- **Hipervëllimi: 5,635,615** (rezultati në Optimize: $-5,635,615.166$), me 40 zgjidhje (fronti ka më pak se 100 pika jo-të-dominuara, prandaj përdoren të gjitha).
- Zgjidhja më ekonomike: $\Delta V = 0.1$ m/s për 2168 ditë.
- Zgjidhja më e shpejtë: 1690 ditë për $\Delta V = 2109$ m/s.
- Krahasuar me renditjen zyrtare të SpOC 2022 (1. Space Coders 5 870 437; 2. fcmaes 5 731 152; 3. Team HRI 5 601 396; 4. haiven 5 503 791), rezultati do të ishte në **vendin e 3-të** nga 20 ekipe.
- Çdo vektor në `results/final/submission.json` është rivlerësuar me funksionin e fitnesit dhe i plotëson të dy kufizimet.

Turet e mira kanë të njëjtën strukturë: hyrje me $T_0 \approx 1500$ ditë dhe $\Delta V_{init} \approx 0$; fly-by i parë kap sondën në një orbitë me periodë 120–200 ditë; fly-by-t e mëpasshëm e zvogëlojnë periodën hap pas hapi (p.sh. $158 \to 79 \to 53 \to 45 \to 34$ ditë), dhe hapat e fundit zgjasin vetëm 20–35 ditë. Planeti c (i dyti nga ylli) shfaqet shpesh si planet i parë: ka masë të madhe dhe është afër yllit, kështu që fly-by i tij e ndryshon shumë orbitën.

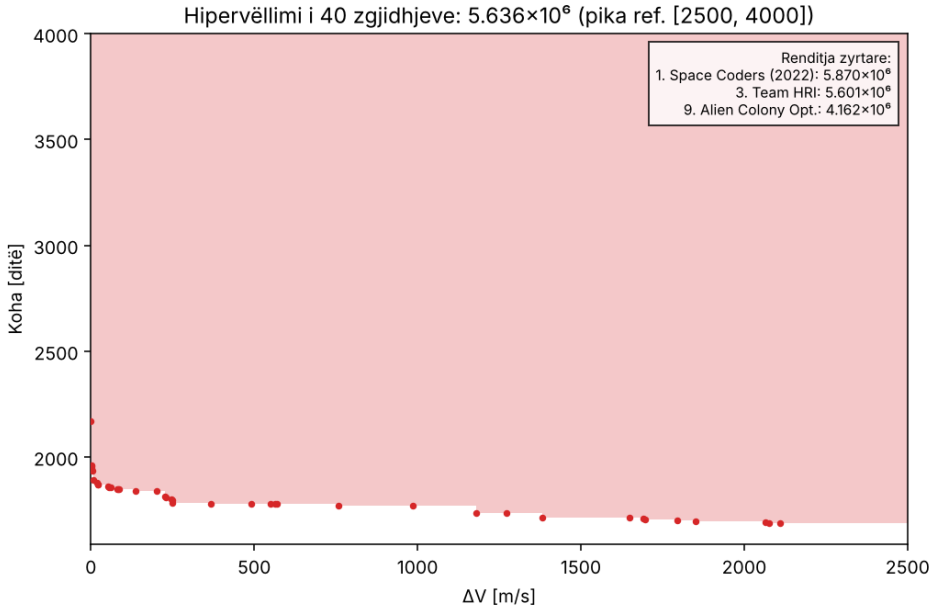


Figura 5: Hipervëllimi i zgjidhjeve të zgjedhura.

8 Përfundime

1. **Analiza e strukturës ia vlen më shumë se fuqia llogaritëse.** Hyrja pothuajse e ndarë, kostot në formë ‘V’ dhe luginat e ngushta shpjegojnë pse algoritmet gjenerike dështojnë dhe drejtuan dizajnin e algoritmit kryesor.
2. **Dekompozimi + beam search** e kthen një problem 34-dimensional të përzier në një varg nënproblemesh 4-dimensionale, me kosto lineare në gjerësinë B dhe polinomiale në numrin e planetëve, në vend të $O(n!)$.
3. **Greedy dështon** sepse vendimet e hershme (hyrja, fly-by i parë) përcaktojnë se çfarë është e mundur më vonë; beam search me *lookahead* zgjidh këtë me kosto të kontrolluar.

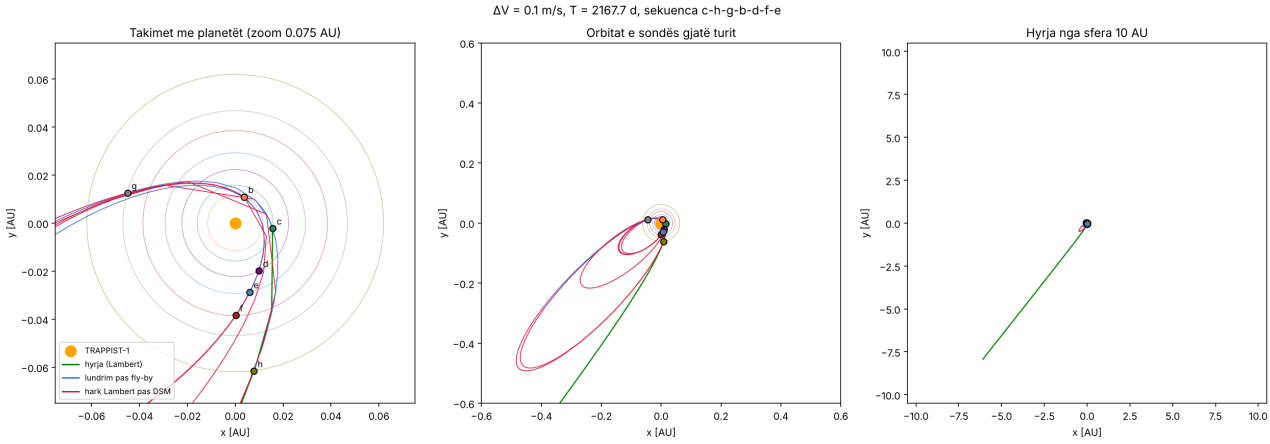


Figura 6: Trajektorja me ΔV minimal.

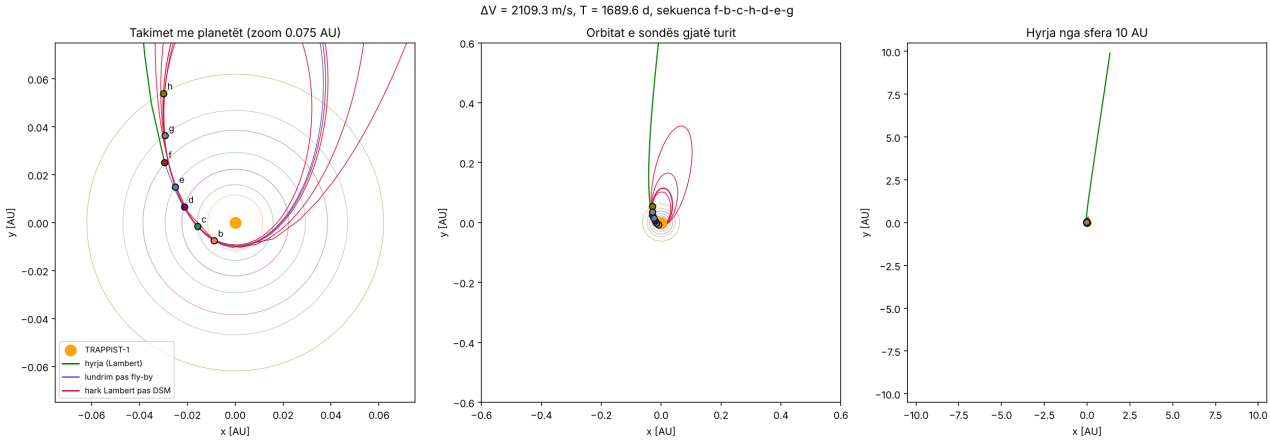


Figura 7: Trajektorja më e shpejtë në front.

4. **Algoritmet ndihmëse** kanë rol të qartë: Lambert-screening (shpejtësi), DE (saktësi në modelin real), CMA-ES (kondicionimi i keq), programimi dinamik (zgjedhja optimale, $O(kn^2)$, e provuar me kërkim shterues).
5. **Vërtetimi i modelit** me integrim numerik të pavarur na lejoi të punojmë pa bibliotekat zyrtare dhe zbuloi veçori të kodit zyrtar (p.sh. periapsi negativ për hiperbola) që ndikojnë drejtpërdrejt në hapësirën e zgjidhjeve.

Punë e ardhshme. (i) Beam më i gjerë ose disa ekzekutime paralele (cilësia rritet me B); (ii) *lookahead* më i saktë, p.sh. kosto minimale e vlerësuar e hapave të mbetur; (iii) optimizim lokal që ruan strukturën (ndryshim i njëkohshëm i kohës dhe i fly-by-t) për të mbushur boshllëqet e frontit; (iv) implementim në C/Numba për 10–50× shpejtësi.

9 Si të ekzekutohet kodi

```
pip install -r requirements.txt
python tests/test_astro.py && python tests/test_vec.py && python tests/test_hv.py
python scripts/run_beam.py --width 150         # A4 (~13 min në 2 bërthama)
python scripts/run_baselines.py --algo nsga2   # A2
python scripts/make_final.py results/runs/beam_*.pkl --out results/final
python scripts/make_figures.py
python scripts/verify_submission.py results/final/submission.json
```

Me `pykep`/`pygmo` të instaluara (conda-forge), `scripts/verify_submission.py --official` rivlerëson çdo zgjidhje edhe me kodin origjinal të ESA-s.

10 Referencat

1. ESA Advanced Concepts Team, *SpOC: Trappist Tour*, https://optimize.esa.int/challenge/spoc-trappist-tour/About.
2. D. Izzo, “Revisiting Lambert’s problem”, *Celestial Mechanics and Dynamical Astronomy* 121, 2015.
3. K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II”, *IEEE TEVC* 6(2), 2002.
4. R. Storn, K. Price, “Differential Evolution – a simple and efficient heuristic for global optimization over continuous spaces”, *J. Global Optimization* 11, 1997.
5. N. Hansen, A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies”, *Evolutionary Computation* 9(2), 2001.
6. K. Bringmann, T. Friedrich, P. Klitzke, “Two-dimensional subset selection for hypervolume and epsilon-indicator”, GECCO 2014.
7. D. Izzo et al., *pykep* (v2.6), https://github.com/esa/pykep.
8. S. Limmer, F. Lanfermann, N. Einecke, “SpOC: approaches by HRI”, SpOC Workshop, ESA 2022.